

Safe Navigation for Ackermann Drive Robots

RBE 575 Final Project

Benjamin Cruse · Pranay Katyal · Josh Ethan Nirmal

Overview

This project implements a safety-guaranteed navigation system for an Amazon DeepRacer, a small Ackermann-drive robot augmented with a SICK TIM781S scanning LiDAR. The platform is modelled as a unicycle with inputs (v, ω) , a valid approximation at the low navigation speeds where the Ackermann slip angle remains small. Rather than replacing the ROS2 Nav2 planner, the CBF layer acts as a safety filter: Nav2 computes a nominal velocity command u_{des} , which is passed to a CBF-QP node that minimally modifies it to satisfy safety constraints before forwarding the filtered command u^* to the robot’s actuators.

Simulation experiments were conducted in a 10×10 m environment with a horizontal wall containing a 2 m gap. A patrol obstacle crosses the gap at 0.8 m/s, requiring the robot to time its crossing while simultaneously avoiding the wall, thus stressing both the static and dynamic CBF layers. The CBF framework provides a forward invariance guarantee: if the system starts inside the safe set $\mathcal{C} = \{x \mid h(x) \geq 0\}$, the control law ensures it remains there for all future time. In practice this guarantee is subject to discrete-time execution at 20 Hz, the unicycle model approximation, and simplified gradient computation.

Three papers were considered during the design process:

1. *Point Cloud-Based CBF Regression for Safe and Efficient Vision-Based Control* [1] : De Sa et al.
2. *Beyond Collision Cones: Dynamic Obstacle Avoidance via Dynamic Parabolic CBFs* [3] : Park et al.
3. *Safe Navigation Under Localization Uncertainty Using Robust CBFs* [2] : Miksits et al.

Miksits et al. was not implemented as it requires a pre-built Euclidean Distance Field (EDF), conflicting with our goal of operating in unknown environments. Methods 1 and 2 were both implemented in simulation. For real-world deployment only Method 1 was used, as Method 2 requires explicit obstacle velocity estimates which would demand a separate tracking pipeline beyond the scope of this project.

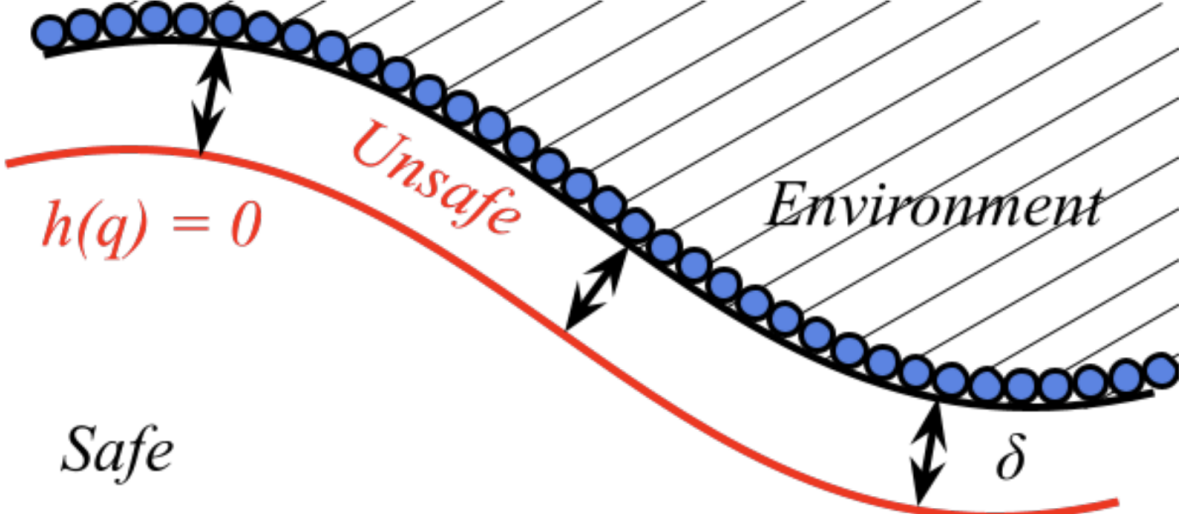
Control Barrier Functions and Math

2.1 Point Cloud-Based CBF (de Sa et al.)

2.1.1 Introduction

De Sa et al. [1] propose Depth-CBF, a computationally lightweight framework for synthesizing control barrier functions directly over point cloud data. Rather than building a metric map of the environment, the method defines safety with respect to the closest point in the live lidar scan, enabling real-time obstacle avoidance in unstructured and dynamically changing environments with no training required. The authors validate the approach on a TurtleBot3 with a lidar sensor, demonstrating safe navigation through cluttered hallways and avoidance of moving obstacles. A

key advantage is computational efficiency, the full pipeline runs at an average of 485 Hz, making it well-suited for resource-constrained platforms.



2.1.2 Barrier Equation and Constraint QP

Safety is defined by the squared distance from the robot to the nearest lidar point, offset by the minimum safe buffer δ .

$$h(q) = \|q - p^*\|^2 - \delta^2$$

The closest point p^* is selected from the live lidar point cloud $\mathcal{P}$.

$$p^* = \arg \min_{p \in \mathcal{P}} \|q - p\|^2$$

The spatial gradient of h with respect to robot position, treating p^* as fixed between scan updates.

$$\nabla h = 2(q - p^*)$$

For the integrator system ($\dot{q} = u$), the drift term vanishes and the input-dependent term equals the gradient directly.

$$\mathcal{L}_f h = 0, \quad \mathcal{L}_g h = \nabla h = 2(q - p^*)$$

Following the cascaded integrator formulation of [1], the QP operates on Cartesian velocity $u = [v_x, v_y]^\top$ rather than unicycle inputs (v, ω) directly, enabling both dimensions to appear in the safety constraints. The QP minimises deviation from the nominal Nav2 command u_{des} while enforcing one safety constraint per nearest obstacle point. Slack variables s_i with penalty λ_s keep the QP feasible when constraints conflict.

$$\begin{aligned} u^* &= \arg \min_{u, s_i \geq 0} \frac{1}{2} \|u - u_{\text{des}}\|^2 + \lambda_s \sum_{i \in \mathcal{K}} s_i \\ \text{s.t.} \quad & 2(q - p_i^*)^\top u \geq -\alpha_s \cdot h_i(q) - s_i \quad \forall i \in \mathcal{K} \\ & -v_{\text{max}} \leq u_j \leq v_{\text{max}}, \quad j \in \{x, y\} \end{aligned}$$

2.1.3 Adaptation and Implementation

The original Depth-CBF computes ∇h via a local mesh regression that fits a quadratic surface to sampled barrier values around the robot, capturing how p^* changes with q [1]. We replace this with the simplified gradient $\nabla h = 2(q - p^*)$, treating p^* as fixed between scan updates. This is valid in our setting because the SICK TIM781S lidar updates at 20 Hz, providing a fresh p^* every 50 ms, fast enough that the error from holding p^* constant remains small relative to the robot’s motion.

The paper also presents single-constraint control as a key contribution, collapsing all point-cloud geometry into one QP constraint via the mesh fit. We instead use $k = 7$ separate constraints, one per nearest scan point, providing better surface coverage without the mesh regression overhead, at the cost of potential constraint conflicts in tight spaces which are handled by slack variables s_i . Regarding the QP decision variable: computing $\mathcal{L}_g h$ for the unicycle system reveals that ω does not appear in the constraint, leaving the barrier relative degree one in v only. Following [1], we therefore run the QP on the integrator $\dot{q} = u$ with $u = [v_x, v_y]^\top$, so that both Cartesian directions appear in the safety constraint.

The safe output u^* is converted back to unicycle inputs via $v^* = \pm \|u^*\|$ and a heading correction term for ω^* . The sign of v^* is determined by projecting u^* onto the robot’s heading vector: if the safe velocity points opposite to the current heading, the robot reverses rather than stopping, an extension not present in the original work.

2.1.4 Tuning Parameters

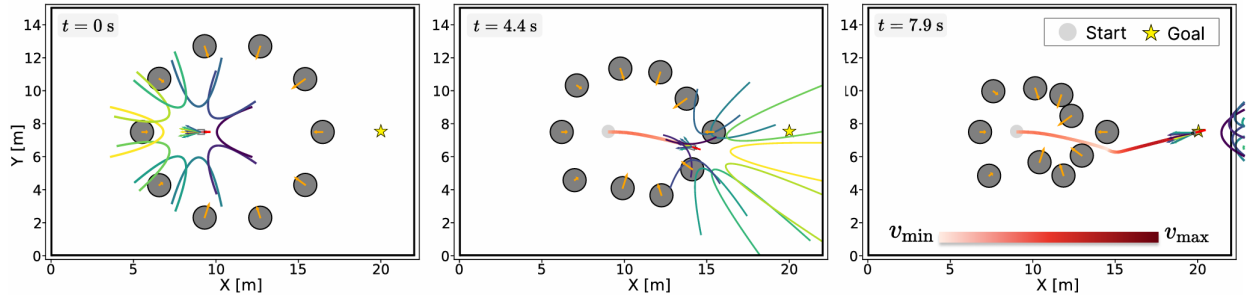
Parameter	Value	Role and effect
α_s	5	Class-K gain on the CBF constraint. Larger values allow the robot to approach obstacles faster before intervening; smaller values enforce earlier, smoother braking.
δ	0.17 m	Safety buffer radius. $h = 0$ when the robot is exactly δ m from the nearest scan point. Set to robot radius plus a small margin.
d_{thresh}	0.7 m	Maximum range at which scan points generate QP constraints. Points beyond this are ignored, giving zero intervention in open space.
k	7	Number of nearest scan points included as constraints per QP solve. More constraints improve geometric coverage but risk contradiction in corners.
λ_s	500	Slack penalty weight. High values treat constraints as nearly hard; lower values allow more relaxation to preserve QP feasibility.
v_{max}	1.0 m/s	Box constraint on Cartesian output velocity, applied to both v_x and v_y , forward and reverse.

Table 1: Tuning parameters for the static Point Cloud-Based CBF.

2.2 Dynamic Parabolic CBF (Park et al.)

2.2.1 Introduction

Park et al. [3] introduce the Dynamic Parabolic Control Barrier Function (DPCBF) to address a fundamental limitation of collision cone-based methods: because cone approaches evaluate only the *heading* of the relative velocity, they prohibit any motion toward an obstacle regardless of distance, leading to excessive conservatism and frequent QP infeasibility in dense environments. DPCBF replaces the fixed cone with a state-dependent parabolic safety boundary whose curvature $\lambda(x)$ and vertex shift $\mu(x)$ adapt online based on both the clearance to the obstacle and the magnitude of the relative velocity. The authors prove validity for the kinematic bicycle model under input constraints and demonstrate a 100% success rate with up to 10 dynamic obstacles, in scenarios where all baseline methods fail due to infeasibility.



2.2.2 Barrier Equation and Constraint QP

The relative position and velocity between the robot and the dynamic obstacle are defined in the global frame.

$$p_{\text{rel}} = \begin{bmatrix} x_{\text{obs}} - x \\ y_{\text{obs}} - y \end{bmatrix}, \quad v_{\text{rel}} = \begin{bmatrix} v_{\text{obs},x} - v_x \\ v_{\text{obs},y} - v_y \end{bmatrix}$$

Coordinates are rotated to the Line-of-Sight (LoS) frame, aligning the $\tilde{x}$ -axis with the vector from the robot to the obstacle.

$$\alpha = \text{atan2}(p_{\text{rel},y}, p_{\text{rel},x}), \quad R(-\alpha) = \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix}$$

$$\tilde{v}_{\text{rel}} = R(-\alpha) v_{\text{rel}} = \begin{bmatrix} \tilde{v}_{\text{rel},x} \\ \tilde{v}_{\text{rel},y} \end{bmatrix}$$

The geometric clearance $d(x)$ measures the available safe distance accounting for the combined robot and obstacle radii r .

$$d(x) = \sqrt{\|p_{\text{rel}}\|^2 - r^2}$$

Two state-dependent design maps adapt the parabola shape online. λ adjusts the curvature and μ shifts the vertex forward by a safe margin.

$$\lambda(x) = k_\lambda \frac{d(x)}{\|v_{\text{rel}}\|}, \quad \mu(x) = k_\mu d(x)$$

The barrier function measures how close the relative velocity endpoint is to the parabolic safe boundary in the LoS frame.

$$h(x) = \tilde{v}_{\text{rel},x} + \lambda(x) \tilde{v}_{\text{rel},y}^2 + \mu(x)$$

The drift term captures how h changes due to obstacle motion alone, with the robot held fixed. It is computed numerically at each timestep rather than analytically, as we use the cascaded integrator formulation instead of the full bicycle model of [3].

$$\mathcal{L}_f h \approx \frac{h(q, p_{\text{obs}} + v_{\text{obs}} \Delta t) - h(q, p_{\text{obs}})}{\Delta t}$$

The input-dependent Lie derivative for the cascaded integrator formulation, where the QP decision variable is the Cartesian velocity $u = [v_x, v_y]^\top$.

$$\begin{aligned} \frac{\partial h}{\partial v_x} &= -\cos \alpha + 2\lambda \tilde{v}_{\text{rel},y} \sin \alpha, & \frac{\partial h}{\partial v_y} &= -\sin \alpha - 2\lambda \tilde{v}_{\text{rel},y} \cos \alpha \\ \mathcal{L}_g h &= \begin{bmatrix} \partial h / \partial v_x \\ \partial h / \partial v_y \end{bmatrix}^\top \end{aligned}$$

The full CBF condition including the drift term ensures h does not decrease faster than α_{dyn} times its current value.

$$\mathcal{L}_f h + \mathcal{L}_g h u \geq -\alpha_{\text{dyn}} \cdot h(x)$$

2.2.3 Adaptation and Implementation

The paper derives DPCBF for the kinematic bicycle model with state $x = [x, y, \theta, v]^\top$ and inputs $u = [a, \beta]^\top$, where the QP directly controls longitudinal acceleration and steering angle. We instead apply the cascaded integrator formulation, running the QP on $u = [v_x, v_y]^\top$ for the same reasons as the static CBF: ω does not appear in $\mathcal{L}_g h$ at first order, making the integrator the natural choice. A critical insight arising from this formulation is that $\mathcal{L}_f h \neq 0$ even for the integrator system. In standard CBF analysis, $\mathcal{L}_f h = 0$ when the drift term $f(x) = 0$.

However, h here depends on the patrol obstacle’s position, which evolves independently of the robot’s control input. As the patrol moves, the Line-of-Sight angle α rotates, changing $\tilde{v}_{\text{rel}}$ and therefore h even when the robot is stationary. This drift term is non-zero and must be included in the CBF condition; omitting it leads to constraint violations when the obstacle approaches a stopped robot. We compute it numerically as $\mathcal{L}_f h \approx [h(q, p_{\text{obs}} + v_{\text{obs}} \Delta t) - h(q, p_{\text{obs}})] / \Delta t$, projecting the patrol one timestep forward while holding the robot fixed.

Two assumptions from the original proof are also violated in our implementation. Assumption 1 requires $v \geq v_{\text{min}} > 0$, excluding the degenerate case where the robot is stationary and $\mathcal{L}_f h = 0$ in the bicycle model. We allow $v = 0$ and address the resulting division by zero in $\lambda(x) = k_\lambda d / \|v_{\text{rel}}\|$ by clamping the relative speed to $\|v_{\text{rel}}\|_{\text{min}} = 0.05 \text{ m/s}$. Assumption 2 requires $\|p_{\text{rel}}\| \geq p_{\text{min}} > r$ at all times, ensuring the robot never enters the obstacle’s footprint. We address this by setting a large detection threshold $d_{\text{DPCBF}} = 15 \text{ m}$, activating the constraint early enough that $h > 0$ at first detection.

Finally, in the current implementation DPCBF activates only when a `/patrol_obstacle` topic is published; on real hardware, extending this to arbitrary dynamic obstacles would require a lidar-based person tracker to estimate obstacle position and velocity online.

2.2.4 Tuning Parameters

Parameter	Value	Role and effect
α_{dyn}	10	Class-K gain for the dynamic constraint. Set higher than the static gain to produce faster reaction to a moving obstacle.
k_{λ}	0.7	Parabola curvature gain ($\lambda = k_{\lambda}d/\ v_{\text{rel}}\ $). Larger values widen the unsafe set, increasing conservatism. The paper’s validated value is 0.144; our value was empirically tuned for our system parameters.
k_{μ}	1.0	Vertex shift gain ($\mu = k_{\mu}d$). Shifts the parabola boundary away from the origin, creating feasible space for motion toward the obstacle. Paper value is 0.505; our higher value provides a larger margin.
$\ v_{\text{rel}}\ _{\text{min}}$	0.05 m/s	Floor on relative speed to prevent division by zero in λ . Motivated by the paper’s Assumption 1 which requires $v > v_{\text{min}} > 0$.
d_{DPCBF}	15.0 m	Detection range for the patrol obstacle. Set large to ensure $h > 0$ at first activation, addressing the paper’s Assumption 2.
Δt	0.02 s	Timestep for numerical $\mathcal{L}_f h$ estimation. Matched to the 50 Hz TF update rate of the system.

Table 2: Tuning parameters for the Dynamic Parabolic CBF. Paper values of $k_{\lambda} = 0.144$ and $k_{\mu} = 0.505$ were derived for a bicycle model with different system parameters; our values were re-tuned empirically.

Implementation

3.1 Hardware

The Deepracer (DR) as shown in Figure 1 was fitted with a SICK TiM-781S LiDAR for obstacle detection and SLAM, which is supported by a custom 3D printed mount designed to fit within the existing features of the DR. The LiDAR is connected to the DR via a USB-A Ethernet adapter. Because the LiDAR requires a voltage greater than the DR batteries could supply (9V-24V vs 5V from USB), a 5S battery was installed to provide power. With all of these additions, the base suspension of the DR became wholly overloaded and were replaced with 3D printed spacers to provide more level driving, improving LiDAR performance.

3.2 Software

3.2.1 Deepracer Data Piping

From the output Ethernet port of the LiDAR, point cloud data received over the USB Ethernet interface on the DR is immediately piped to a local address on the DR’s subnet. This data is then transported over a LAN to a remote computer running the SLAM/CBF/NAV2 Docker container. The container processes the point cloud data into an occupancy grid and computes an updated

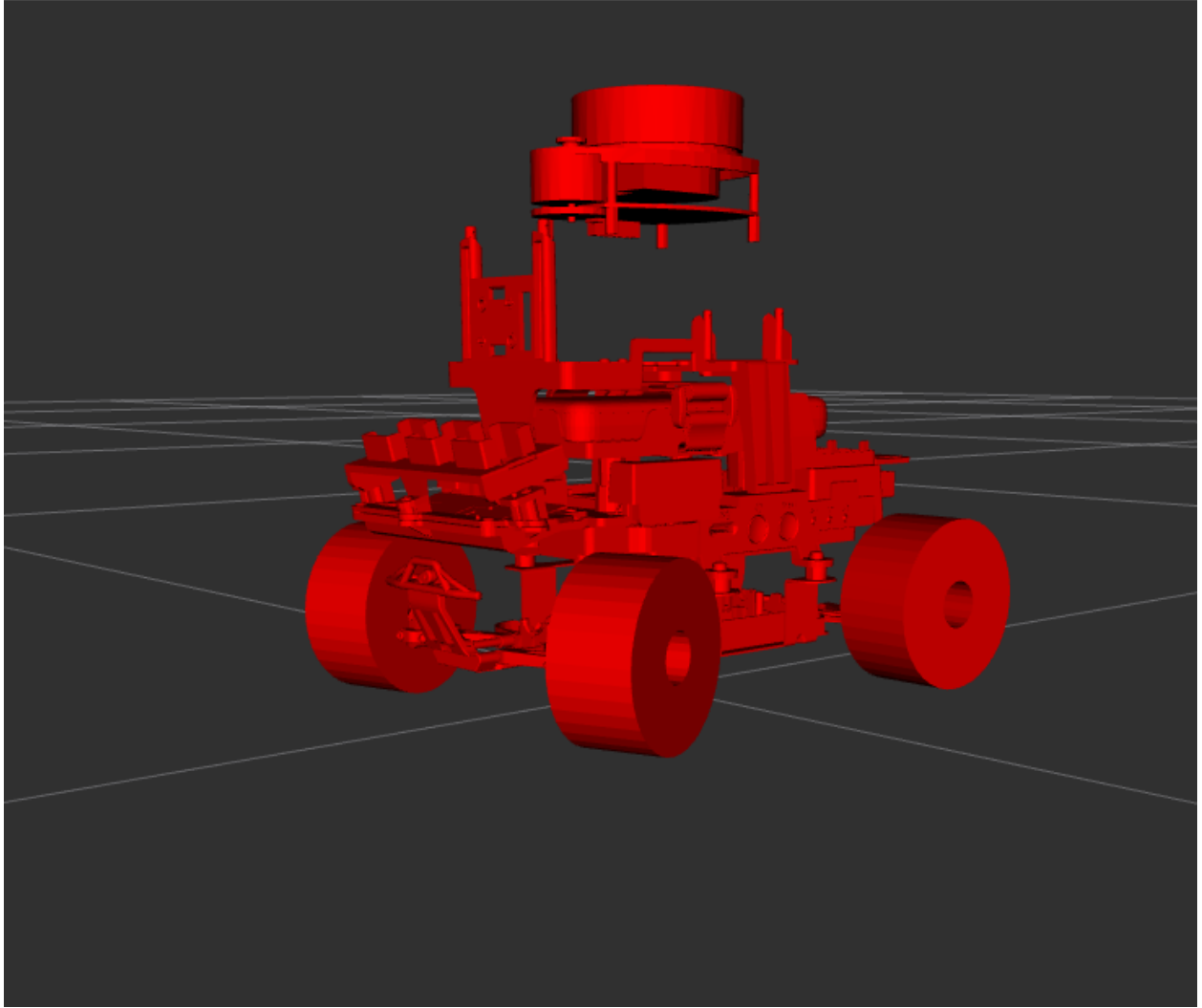


Figure 1: The model used in the RViz simulation.

pose estimate based on the current pass of point cloud data. In addition, distance calculations are performed to select a set of points to center CBFs around. Once a target pose is selected using Nav2 in RViz, the resulting twist is fed in as the nominal control value to the QP constructed with the CBFs. The 'safe' twist is then sent to the DR to execute through its throttle library.

3.3 Remote Docker Container

3.3.1 Odometry, SLAM toolbox, and Nav2

Twist based odometry quickly fails due to the differences between published twist values and executed throttle commands. As such, we rely on LiDAR odometry utilizing ROS 2's RF2O package. The odometry data is fed into the SLAM node utilizing the SLAM toolbox package, and a resulting local occupancy grid is formed.

When the container is launched, an RViz instance opens and begins to visualize the map generated from LiDAR data. A goal pose can then be set within and Nav2 will attempt to generate a path

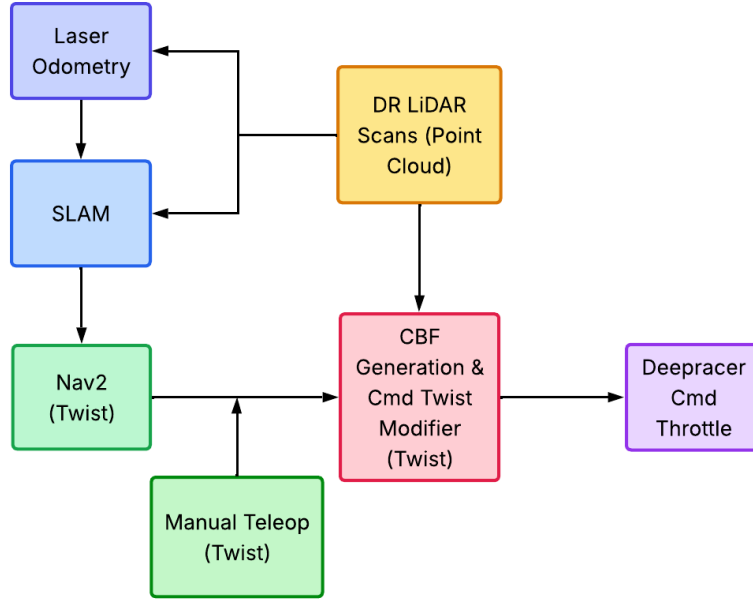


Figure 2: The flow of data through the pipeline

using Ackermann dynamics to the final destination.

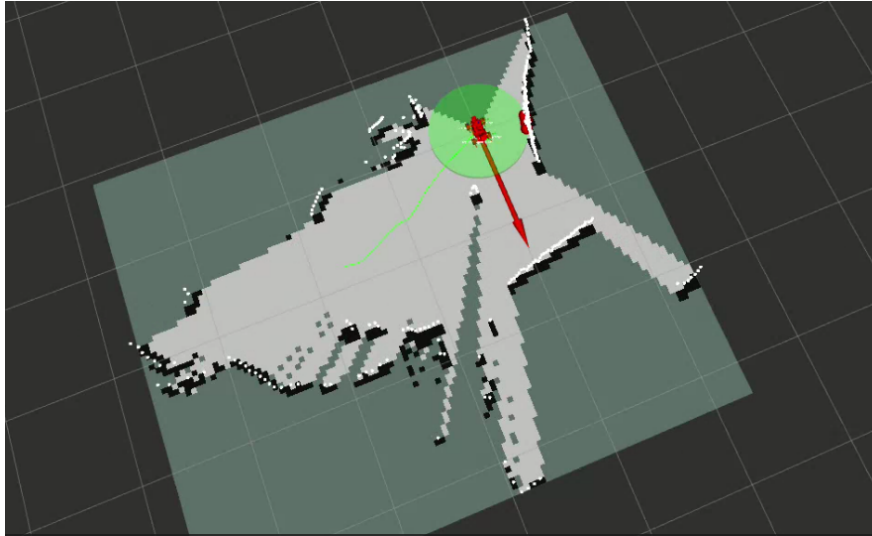


Figure 3: RViz Visualization of environment (Nav2 path points as green lines, CBF zone as green circle, points selected for CBF in red, and LiDAR points in white)

3.3.2 Twist and LiDAR Bridge

To facilitate data transmission to and from the Deepracer, two ROS2 nodes are created: One listening on port 3001 for incoming LiDAR data and the other transmitting the safe twists to the Deepracer on port 5002. The first node publishes all received data to the `/scans` topic while the second transmits all twists published to the `/cmd_vel.safe` topic.

3.3.3 CBF Node

From the entire point cloud, the CBF node culls all points falling outside of a distance threshold to simplify calculations. From this subset, the node finds the point closest to the DR, culls all points within a defined radius, and then repeats this N more times. The selected points are then published to `/cbf_markers` for visualization. From the resulting N points, a CBF is created on each point with a barrier to keep the DR at least distance D from each. New CBFs are generated 50 times a second from incoming LiDAR data. Whenever a twist is published to `/cmd_vel`, the node minimizes a QP from the generated CBFs with the `/cmd_vel` twist as its nominal control output. It then outputs the safe twist to the `/cmd_vel_safe` topic.

3.4 Sim Docker Container

The sim Docker container operates very similarly to the hardware container, albeit simulating LiDAR readings and running a patrolling obstacle to demonstrate the dynamic CBF. LiDAR is simulated by raycasting from the center of the DR and generate scan points when an occupied tile on the occupancy grid is encountered.

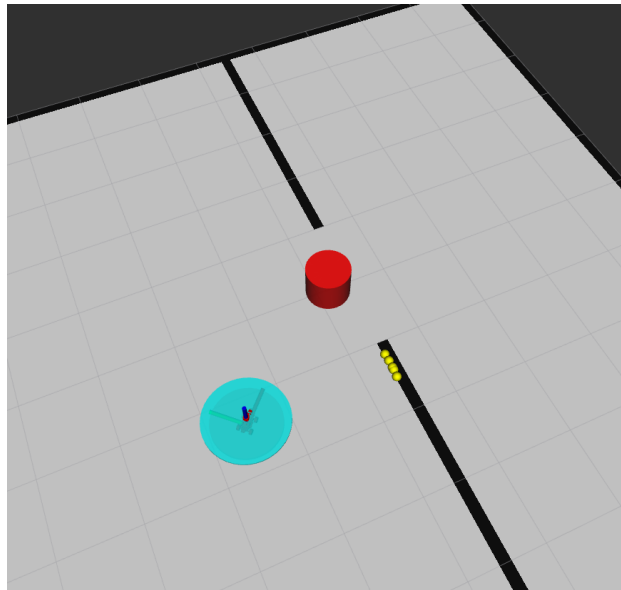


Figure 4: A simulation of the Deepracer in an environment with a moving obstacle.

3.5 Deepracer

3.5.1 LiDAR Transmission

Because SICK LiDAR transmits its data via Ethernet, an alias had to be created to allow the socket layer to receive and parse the data. This alias mapped all incoming packets from the LiDAR's output address to another IP address and port available locally on the Deepracer. This may have resulted in several downstream socket layer issues, but those issues appeared to persist even after all networking settings were reset to default, so there is no strong evidence either way. These packets are transmitted to port 3001 on the remote computer.

3.5.2 Twist Reception

Once the safe twists are created, they are then transmitted to the Deepracer on port 5002. Because of the previously insinuated socket layer issues, direct UDP transmission to a ROS2 node or Python program became impossible. To get around this, a ROS2 node that listens to incoming TCP data on port 5002, where the received packets are then manually reconstructed into twists for the Deepracer to execute. Recently, for unknown reasons, the UDP twist bridge occasionally fails to publish to `/cmd_vel` for the Deepracer despite correctly parsing the desired twist from the script. This may be a result of a broken ROS2 environment or some other unknown conflict.

3.5.3 Throttle Control

The default packages for the AWS Deepracer were used for motor control. There are several unexplained issues in converting twists to throttle that will have to be explored. For now, the primary issue to be solved is the behavior determining what magnitude of twist is required to initialize throttle commands and determining why minimum throttle appears to be stuck at 80%.

Conclusion and Results

4.1 Results

4.1.1 Simulation

The simulation scenario placed the robot on one side of a 10×10 m room divided by a horizontal wall with a 2 m gap, with a patrol obstacle crossing the gap at 0.8 m/s. The robot was commanded to navigate through the gap to a goal on the far side using Nav2, with the CBF-QP layer filtering all velocity commands.

The robot successfully navigated through the gap in all tested runs. The static CBF prevented all wall collisions throughout — the robot never made contact with the wall despite being commanded toward it by Nav2 at various approach angles. The DPCBF layer demonstrated the intended reversing behaviour: when the patrol obstacle approached the gap while the robot was attempting to cross, the robot backed away sufficiently to allow the patrol to pass before resuming its approach. This behaviour arises from the negative $\mathcal{L}_f h$ term dominating the CBF constraint, forcing the QP to select a velocity that increases separation from the patrol.

In some runs the patrol obstacle entered the robot’s safety bubble $\delta = 0.17$ m without a collision being observed. This is consistent with the known violation of Assumption 2 from [3], which requires $\|p_{\text{rel}}\| > r$ at all times. When the patrol closes faster than the QP can respond at 20 Hz, the safety boundary can be momentarily breached. The slack variables absorbed these constraint violations, preventing the QP from becoming infeasible, though at the cost of a weakened formal guarantee in those instants.

The nominal Nav2 command u_{des} and the filtered output u^* were visually distinct during obstacle interactions, confirming that the CBF layer was actively modifying commands. A quantitative comparison of command divergence was not recorded in this study and is identified as a direction for future work.

4.1.2 Real Hardware

On the physical DeepRacer, the static CBF successfully prevented collisions with detected obstacles. The SICK TIM781S lidar provided a live point cloud which the CBF node processed at 20 Hz, selecting the $k = 7$ nearest returns within 0.7 m and generating QP constraints accordingly. The robot consistently slowed and redirected when approaching walls, with the RViz2 visualization confirming that the safety bubble markers activated correctly as obstacles entered the detection range.

The DPCBF layer was not active during real hardware tests as no `/patrol_obstacle` publisher was running. Dynamic obstacles such as people in the environment were treated as static scan points by the static CBF only. Full dynamic obstacle avoidance on real hardware would require a lidar-based person tracker as discussed in Section 2.2.

End-to-end autonomous navigation on hardware was partially demonstrated, with remaining issues in the `cmd_vel` relay and servo throttle initialization preventing a complete autonomous run at the time of submission.

4.2 Conclusion

While we were able to successfully implement a CBF pipeline that manipulates target twist commands to adhere to safety specifications, we were unable to get the DeepRacer to operate as consistently as hoped. In the future, we will examine the AWS DeepRacer packages and attempt to find explanations for the currently unknown faults in the pipeline. We will attempt to determine where the issues in the DeepRacer `/cmd_vel` publisher originate and create a permanent fix.

4.3 Github

<https://github.com/pranaykatyal/safetybarrierondeepracer>

References

- [1] Massimiliano De Sa, Prasanth Kotaru, and Koushil Sreenath. Point cloud-based control barrier function regression for safe and efficient vision-based control. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 366–372, 2024.
- [2] Adam Miksits, Fernando S. Barbosa, Magnus Lindhé, José Araújo, and Karl H. Johansson. Safe navigation of networked robots under localization uncertainty using robust control barrier functions. In *2023 62nd IEEE Conference on Decision and Control (CDC)*, pages 6064–6071, 2023.
- [3] Hun Kuk Park, Taekyung Kim, and Dimitra Panagou. Beyond collision cones: Dynamic obstacle avoidance for nonholonomic robots via dynamic parabolic control barrier functions, 2026.